



Continuous Integration with CodeBuild

CH

Arpita Chowdhury

```
! buildspec.yml
1  version: 0.2
2
3  phases:
4    install:
5      runtime-versions:
6        java: corretto8
7    pre_build:
8      commands:
9        - echo Initializing environment
10       - export CODEARTIFACT_AUTH_TOKEN=`aws codeartifact get-authorization-token --domain nextwork --domain-owne
11    build:
12      commands:
13        - echo Build started on `date`
14        - mvn -s settings.xml compile
15    post_build:
16      commands:
17        - echo Build completed on `date`
18        - mvn -s settings.xml package
19  artifacts:
20    files:
21      - target/nextwork-web-project.war
22    discard-paths: no
```



Introducing Today's Project!

In this project, I will demonstrate how to create and configure an AWS CodeBuild project from scratch, securely connect it to a GitHub repository, define a repeatable build process using a `buildspec.yml` file, and automate testing as part of the build pipeline. I'm doing this project to learn how real-world teams implement continuous integration (CI) by automatically building and testing code on every change, reducing manual effort, catching issues early, and ensuring consistent, reliable deployments using AWS-managed DevOps services.

Key tools and concepts

Services I used were AWS CodeBuild, Amazon S3, AWS IAM, GitHub Key concepts I learnt include continuous integration (CI), build automation with CodeBuild, defining build workflows using `buildspec.yml`, IAM roles and permissions for CI services, artifact storage in S3 and integrating GitHub with AWS using managed connections for secure, production-ready pipelines.

Project reflection

This project took me approximately 4 hours to complete from start to finish. The most challenging part was troubleshooting multiple real-world CI/CD issues, including failed builds due to a missing `buildspec.yml`, resolving IAM permission errors that prevented CodeBuild from accessing CodeArtifact, handling authentication tokens, and fixing SSH connection drops during setup and debugging. It was most rewarding to see the final build succeed end-to-end with automated tests running and artifacts uploaded to S3 because it proved I can design, debug, and operate a complete, production-style CI pipeline.

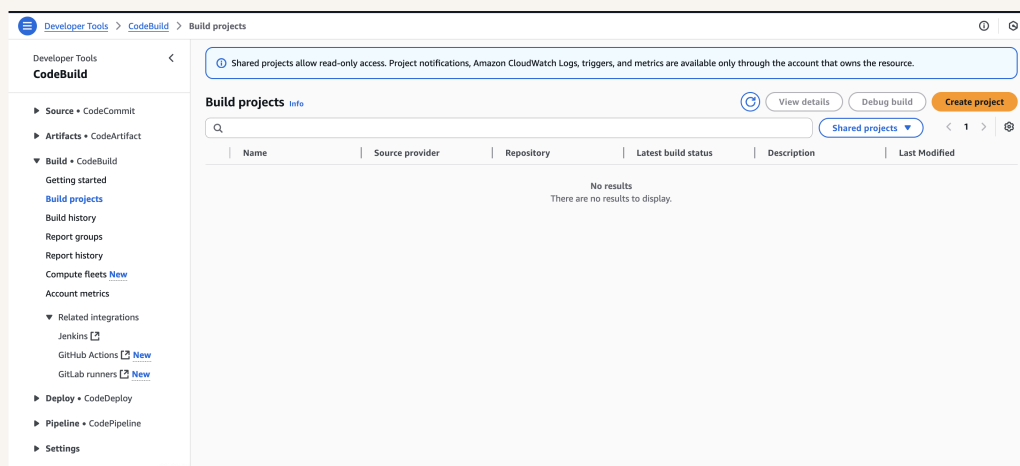
This project is part four of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project Deploy a Web App with CodeDeploy.



Setting up a CodeBuild Project

CodeBuild is a continuous integration service, which means it automatically builds and tests code whenever changes are made, without requiring developers to manage or maintain their own build servers. Engineering teams use it because it provides a fully managed, scalable, and secure way to run builds on demand, catch errors early, and ensure consistent build results as part of a CI/CD pipeline.

My CodeBuild project's source configuration means CodeBuild is directly connected to my application's source repository and automatically pulls the latest code whenever a build is triggered. I selected a Default project type with a direct GitHub source integration so the entire continuous integration process—including fetching source code, running the build steps, and producing build artifacts is fully managed within AWS CodeBuild.

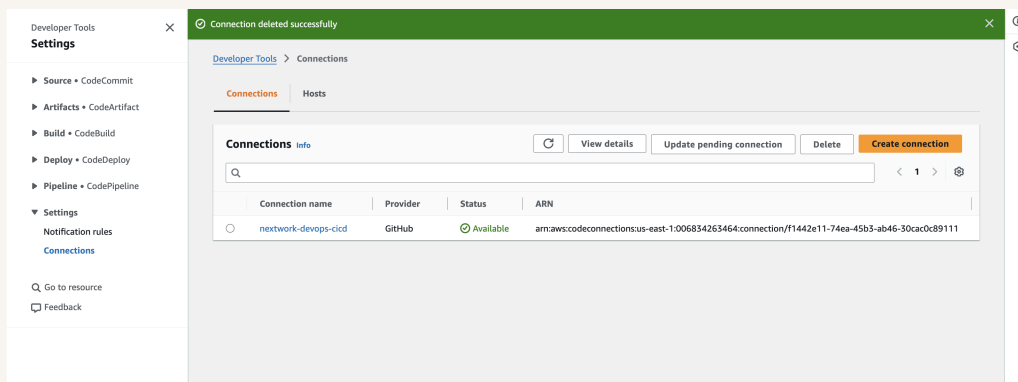




Connecting CodeBuild with GitHub

There are multiple credential types for GitHub, like GitHub App, Personal Access Token, and OAuth App, each offering different levels of security and management complexity. I used GitHub App because it is the most secure and easiest option AWS manages the authentication and permissions automatically, reducing the risk of exposed credentials and eliminating the need to manually create, rotate, or store access tokens.

The service that helped connect my AWS account to my GitHub repository is AWS CodeConnections, which provides a secure, managed way for services like CodeBuild to access private GitHub repositories without manually handling credentials.





CodeBuild Configurations

Environment

My CodeBuild project's Environment configuration means I defined how and where my builds will run, including the compute resources, operating system, and runtime used during the build process. It includes settings like the provisioning model (on-demand), managed build image, EC2-based container compute, Amazon Linux operating system, standard runtime, and a prebuilt CodeBuild image with Corretto (Java) installed, along with an IAM service role that grants CodeBuild permission to access required AWS services during the build.

Artifacts

Build artifacts are the output files produced at the end of a build process, such as compiled binaries, JAR/WAR files, or packaged application files. They're important because they represent the final, deployable version of the application and can be reused, tested, or deployed without rebuilding the code. My build process will create packaged application artifacts generated by the Maven build. To store them, I created an Amazon S3 bucket configured as the artifact store for my CodeBuild project so the build outputs are securely saved and accessible after each build.

Packaging

When setting up CodeBuild, I also chose to package artifacts in a ZIP file, because it bundles all build outputs into a single, compressed archive that is easy to store, transfer, download, and deploy in later stages of a CI/CD pipeline.

Monitoring

For monitoring, I enabled Amazon CloudWatch Logs, which is a managed logging service that collects, stores, and displays real-time build logs from CodeBuild.



buildspec.yml

My first build failed because AWS CodeBuild could not find a buildspec.yml file in the root of my GitHub repository, which caused a YAML file error during the source download phase. A buildspec.yml file is needed because it defines the exact commands and phases CodeBuild must execute such as installing dependencies, running tests, building the application, and producing artifacts so without it, CodeBuild doesn't know how to run the build.

The first two phases in my buildspec.yml file (install + pre_build) set up the build environment and authentication. In install, CodeBuild installs the required Java runtime (Corretto 8). In pre_build, I generate and export a temporary CodeArtifact authorization token so Maven can securely download dependencies and plugins during the build. The third phase in my buildspec.yml file (build) performs the main build work by running mvn clean install using my settings.xml, which compiles the application, runs any configured tests, and produces build outputs. The fourth phase in my buildspec.yml file (post_build) finalizes the process by packaging the application and preparing the deployable artifact. It runs mvn package and outputs the .war file (from target/*.war) so CodeBuild can upload it as a build artifact to S3.



```
! buildspec.yml
1  version: 0.2
2
3  phases:
4    install:
5      runtime-versions:
6        java: corretto8
7    pre_build:
8      commands:
9        - echo Initializing environment
10       - export CODEARTIFACT_AUTH_TOKEN=`aws codeartifact get-authorization-token --domain nextwork --domain-owner
11    build:
12      commands:
13        - echo Build started on `date`
14        - mvn -s settings.xml compile
15    post_build:
16      commands:
17        - echo Build completed on `date`
18        - mvn -s settings.xml package
19  artifacts:
20    files:
21      - target/nextwork-web-project.war
22  discard-paths: no
```



Success!

My second build also failed, but with a different error that said CodeBuild hit a `COMMAND_EXECUTION_ERROR` in the `PRE_BUILD` phase while running the command to generate a CodeArtifact authorization token (`aws codeartifact get-authorization-token`), and it exited with status 254. To fix this, I updated the CodeBuild service role to include the required CodeArtifact permissions (so it can request an auth token and read from the repository), and I also ensured the `buildspec` uses the correct region/account values so the token request succeeds.

To resolve the second error, I updated the CodeBuild service role to include the required AWS CodeArtifact permissions so CodeBuild could successfully run `aws codeartifact get-authorization-token` during the `PRE_BUILD` phase and authenticate to the private repository. When I built my project again, I saw a successful build status, with every phase showing Succeeded (`DOWNLOAD_SOURCE`, `INSTALL`, `PRE_BUILD`, `BUILD`, `POST_BUILD`, and `UPLOAD_ARTIFACTS`), confirming the pipeline could now pull dependencies from CodeArtifact and complete the Maven build end-to-end.

To verify the build, I checked the Amazon S3 bucket configured as the artifact store for my CodeBuild project. Seeing the artifact tells me the build completed successfully and CodeBuild was able to package the application output and upload it to S3, confirming the entire CI process from source code to build artifacts worked as expected.



✔ Build started

You have successfully started the following build: nextwork-devops-cicd:158225d3-d30d-45a6-b1c9-60fd4061c35f

×

nextwork-devops-cicd:158225d3-d30d-45a6-b1c9-60fd4061c35f

Stop buildDebug buildRetry build

Build status

Status

✔ Succeeded

Initiator

arpita-IAM-Admin

Build ARN

arn:aws:codebuild:us-east-1:006834263464:build/nextwork-devops-cicd:158225d3-d30d-45a6-b1c9-60fd4061c35f

Resolved source version

17c5cc5d0cf0bd4a0d6910fa46f6b3816b0fa772

Start time

Jan 7, 2026 10:30 PM (UTC-6:00)

End time

Jan 7, 2026 10:31 PM (UTC-6:00)

Build number

3



Automating Testing

In a project extension, I'm adding automated testing to my CI pipeline to validate the application before completing the build. I added a test script that checks the basic project structure (ensuring the src directory exists), verifies required web application files like index.jsp are present, and runs a simple validation test so the build fails early if core components are missing.

To add the test script to the build process, I included it in my repository and updated the buildspec.yml file to execute the script during the build phase, ensuring the tests automatically run as part of every CodeBuild run and fail the build if any required checks do not pass.

After pushing my code to GitHub, I ran the CodeBuild project (triggering a new build). I could see CodeBuild automatically pulling the latest commit, executing the build and test steps defined in buildspec.yml, and completing successfully with logs in CloudWatch and the build artifact uploaded to S3.



```
$ run-tests.sh M
$ run-tests.sh
1  #!/bin/bash
2
3  echo "==== RUNNING SIMPLE TESTS ===="
4  echo "Test 1: Checking project structure..."
5  if [ -d "src" ]; then
6      echo "✅ PASS: src directory exists"
7  else
8      echo "❌ FAIL: src directory not found"
9      exit 1
10 fi
11
12 echo "Test 2: Checking for web app files..."
13 if [ -f "src/main/webapp/index.jsp" ]; then
14     echo "✅ PASS: index.jsp exists"
15 else
16     echo "❌ FAIL: index.jsp not found"
17     exit 1
18 fi
19
20 echo "Test 3: Simple validation test..."
21 echo "✅ PASS: This test always passes"
22
23 echo "==== ALL TESTS PASSED ===="
24 exit 0
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

